

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería

Departamento de Ciencias de la Computación

CC3092 – Deep Learning

Ing. Luis Suriano



Laboratorio 7

Forecasting de la temperatura del aceite de transformadores
con LSTM y Transformer

Ricardo Arturo Godínez Sánchez - 23247

Vianka Vanessa Castro Ordoñez - 23201

GUATEMALA, 26 de septiembre de 2026

Introducción

Este laboratorio construye un sistema de predicción de series de tiempo sobre el dataset **ETTh1** (*Electricity Transformer Temperature*), que registra cada hora, durante dos años, la temperatura del aceite de transformadores de potencia en China. La variable objetivo es **OT** (*Oil Temperature*): anticipar su valor permite detectar si un transformador está en riesgo de falla térmica.

Con las últimas 96 horas de OT se predice su valor 24 y 48 horas después. Se comparan dos arquitecturas escritas con tensores de PyTorch, sin usar `nn.LSTM` ni `nn.MultiheadAttention`: un **LSTM many-to-one** y un **Transformer encoder** con self-attention. Como referencia se usa un **baseline naive**, que supone que la temperatura dentro de H horas será igual a la actual. Un modelo solo aporta valor si su error es menor que el del naive.

El código está organizado en un notebook por task (`app/Lab7_Task1.ipynb` a `Lab7_Task4.ipynb`). Cada notebook incluye la celda de verificación oficial del enunciado, y todas imprimen **CORRECTO**.

Task 1 · Preprocesamiento y análisis de autocorrelación

1.1 Pipeline de datos

Se descargó ETTh1 desde el repositorio oficial: 17,420 observaciones horarias de 7 variables. Se tomó OT como serie objetivo y se normalizó con *z-score* usando su media y desviación estándar ($\mu = 13.32$ °C, $\sigma = 8.57$ °C). Ambos valores se guardan para desnormalizar: un error de 0.2 en escala normalizada equivale a $0.2 \times 8.57 \approx 1.7$ °C.

La serie se dividió en tres bloques temporales contiguos, sin mezclar observaciones: 60% entrenamiento (10,452 horas), 20% validación (3,484) y 20% prueba (3,484). En series de tiempo esto es indispensable: barajar los datos permitiría al modelo entrenar con información del futuro para predecir el pasado, lo que daría resultados artificialmente buenos.

Después se implementó `make_windows(series, L, H)`, que desliza una ventana de $L = 96$ horas (cuatro días) sobre cada split. Cada ventana es una entrada y su objetivo es el valor que aparece H horas después del último elemento:

$$X^{(t)} = (x_{t-L+1}, \dots, x_t), \quad y^{(t)} = x_{t+H}$$

Split	Ventanas H = 24	Ventanas H = 48
Entrenamiento	10,333 × 96	10,309 × 96
Validación	3,365 × 96	3,341 × 96
Prueba	3,365 × 96	3,341 × 96

Tabla 1. Tamaño de los conjuntos de ventanas.

1.2 Función de autocorrelación (ACF)

La ACF mide cuánto se parece la serie a sí misma desplazada k horas. Se implementó `acf_manual` con la fórmula del enunciado y se calculó sobre el conjunto de entrenamiento hasta el lag 96, con bandas de confianza al 95% en $\pm 1.96/\sqrt{N} = \pm 0.019$.

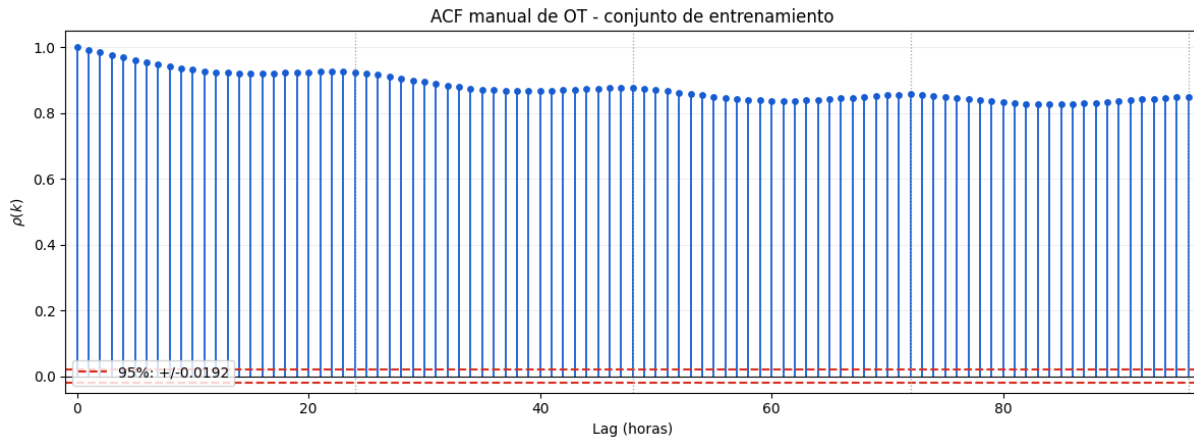


Figura 1. ACF de OT sobre el conjunto de entrenamiento (lags 0 a 96). Las líneas punteadas grises marcan los múltiplos de 24 horas y las rojas, las bandas de confianza al 95%.

Toda la ACF queda muy por encima de las bandas de confianza: la temperatura del aceite es una serie muy persistente, y su valor en una hora depende fuertemente de las anteriores.

1.2a. Identifique el lag con el segundo pico más alto de la ACF (después de lag 0). Interprete ese valor en términos de la periodicidad de la temperatura del aceite.

Después del pico trivial en el lag 0, el primer pico local aparece en el **lag 22**, con $\rho(22) \approx 0.926$. La zona entre 22 y 24 horas es prácticamente plana ($\rho(24) \approx 0.925$), así que este pico representa un **ciclo diario**: la temperatura a cierta hora se parece mucho a la del día anterior a la misma hora, siguiendo el patrón de carga eléctrica entre el día y la noche. El corrimiento de 22 a 24 horas se explica porque el ciclo térmico real no es perfectamente periódico ni sinusoidal. También hay picos locales cerca de 47 y 72 horas ($\rho \approx 0.877$ y 0.857), que son múltiplos del mismo ciclo diario.

1.2b. Con base en el correlograma obtenido, justifique si $L = 96$ es una elección apropiada para la longitud de ventana o si debería ser mayor o menor.

$L = 96$ es una elección apropiada. La ventana cubre cuatro días completos, así que contiene varios ciclos diarios que el modelo puede comparar entre sí. La dependencia sigue siendo fuerte en todos los múltiplos del ciclo: $\rho(24) \approx 0.925$, $\rho(48) \approx 0.876$, $\rho(72) \approx 0.857$ y $\rho(96) \approx 0.849$. Una ventana menor que 48 horas perdería ciclos útiles. Como $\rho(96)$ todavía es alto, una ventana mayor podría evaluarse, pero con la ACF calculada solo hasta 96 no hay evidencia para decidir cuánto mayor. Por eso 96 es un compromiso razonable entre contexto y costo computacional.

Baseline naive y verificación

El baseline naive predice $\hat{y} = x_t$: la temperatura futura igual a la actual. En validación obtiene un MAE de **0.1996** para $H = 24$ y **0.2604** para $H = 48$. Es una referencia exigente precisamente por la alta persistencia de la serie. La verificación oficial del Task 1 pasó correctamente ($ACF(24) = 0.9250$ y $MAE \text{ naive} = 0.1996$, ambos dentro de los rangos esperados).

Task 2 · LSTM implementado manualmente

2.1 La celda LSTM

El LSTM lee la ventana hora por hora y mantiene dos estados: la **celda** c , una memoria de largo plazo, y el **estado oculto** h , un resumen de lo visto hasta ese momento. En cada paso, cuatro compuertas deciden cómo actualizar la memoria:

- **Olvido (f)**: cuánto de la memoria anterior se conserva.
- **Entrada (i)**: cuánto de la información nueva se deja entrar.
- **Candidato (g)**: qué información nueva se propone.
- **Salida (o)**: cuánto de la memoria se expone en h .

$$c_t = f_t \cdot c_{t-1} + i_t \cdot g_t \quad h_t = o_t \cdot \tanh(c_t)$$

Los pesos de las cuatro compuertas se declararon como `nn.Parameter` apilados (W_x : 128×1 , W_h : 128×32), y el `forward` recorre explícitamente los 96 pasos con un ciclo `for`. Es un modelo *many-to-one*: solo el último estado oculto se proyecta a la predicción escalar.

2.2 Entrenamiento

Se entrenaron dos modelos ($H = 24$ y $H = 48$) con $d_h = 32$, Adam, 20 épocas y *batch size* 64. El primer intento, que predecía directamente la temperatura, obtuvo un MAE de 0.300 en validación, **peor que el naive**. La causa es un cambio de nivel entre splits: la serie normalizada tiene media +0.46 en entrenamiento y -0.74 en validación, así que el modelo aprendía las temperaturas típicas de entrenamiento y fallaba al cambiar de nivel.

La solución fue la **predicción residual**. El modelo recibe la ventana relativa a su último valor y predice cuánto va a cambiar la temperatura; al final se suma de vuelta ese último valor:

$$\hat{y} = x_t + f(X - x_t)$$

Si el modelo no aprendiera nada, la predicción sería exactamente el naive. Todo lo que aprende es una corrección sobre esa base, y los patrones de cambio sirven en cualquier nivel de temperatura. La clase del modelo no cambió: solo cambiaron la entrada y el objetivo del entrenamiento.

2.2a. Use la función de pérdida que considere apropiada para este problema y justifique su elección.

Se usó el **error cuadrático medio (MSE)** por cuatro razones:

1. **Es la pérdida natural para predecir un valor continuo.** Minimizarla equivale a suponer ruido gaussiano, y su óptimo es el valor esperado del futuro dada la ventana: la mejor predicción puntual.
2. **Castiga más los errores grandes.** Como el error se eleva al cuadrado, equivocarse por 5 °C pesa cuatro veces más que equivocarse por 2.5 °C. En este problema es deseable: subestimar un pico de temperatura es justamente el caso que puede terminar en una falla térmica.
3. **Entrena de forma estable.** Su gradiente es suave y proporcional al error, a diferencia del MAE, cuyo gradiente tiene magnitud constante y no está definido en cero. Además es coherente con el RMSE que

se reporta.

4. **Los datos están normalizados**, así que la pérdida cuadrática no explota numéricamente.

Como limitación, el cambio de temperatura que se predice tiene colas pesadas (curtosis de 5.6 para $H = 24$, contra 3 de una distribución normal), y el MSE es sensible a esos cambios extremos; una alternativa sería la pérdida de Huber. Se mantuvo el MSE porque esos cambios extremos son los eventos de riesgo que interesa no subestimar.

Resultados en validación

Modelo	H	MAE	RMSE	MAE en °C	Ratio vs naive
LSTM	24 h	0.1959	0.2602	1.68	0.982
LSTM	48 h	0.2389	0.3096	2.05	0.918

Tabla 2. Métricas del LSTM en validación (escala normalizada, salvo indicación).

Ambos modelos superan al naive (ratio menor que 1). La verificación oficial del Task 2 pasó correctamente: la salida tiene la forma esperada y el MAE con $H = 24$ es menor que 1.1 veces el del naive.

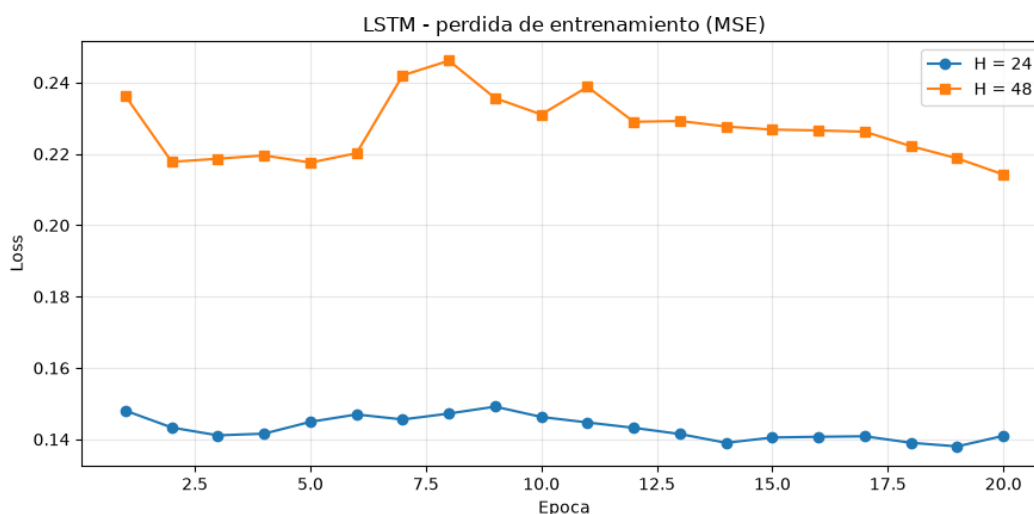


Figura 2. Pérdida de entrenamiento (MSE) del LSTM por época para ambos horizontes.

La pérdida con $H = 48$ es mayor durante todo el entrenamiento, porque predecir más lejos es más difícil. En ambos casos baja poco a poco: el modelo parte de una base fuerte (el naive) y aprende correcciones pequeñas.

Task 3 · Transformer con self-attention implementado manualmente

3.1 Positional encoding

La atención, por sí sola, no conoce el orden de los datos: si se revuelven las 96 horas, ve lo mismo. Por eso cada valor escalar se proyecta primero a un vector de 32 dimensiones con `nn.Linear(1, d_model)` y se le suma un **positional encoding** sinusoidal: una huella única de senos y cosenos de distintas frecuencias para cada posición. Se registró como un *buffer*, así que no se entrena.

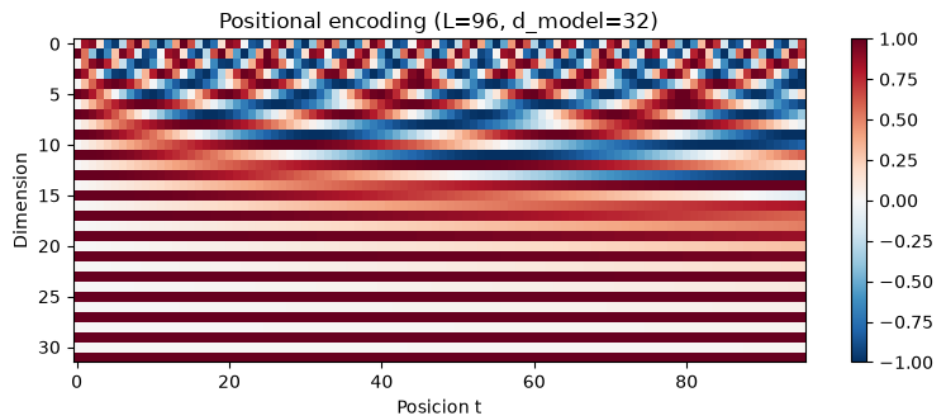


Figura 3. Positional encoding para $L = 96$ y $d_{\text{model}} = 32$. Las primeras dimensiones oscilan rápido y las últimas lento, lo que permite distinguir cada posición.

3.2 Multi-head self-attention

Cada posición genera tres vectores: una **query** (qué busca), una **key** (qué ofrece) y un **value** (qué información entrega). La compatibilidad entre cada par de posiciones se calcula comparando queries con keys, se convierte en pesos con *softmax* (cada fila suma 1) y con esos pesos se promedian los valores:

$$\text{Atención}(Q, K, V) = \text{softmax}(Q K^T / \sqrt{d_k}) \cdot V$$

Se usaron dos cabezas en paralelo, cada una con sus propias proyecciones, que después se concatenan y se proyectan con W_O . El bloque completo sigue el orden del enunciado: atención, *Add & Norm* (conexión residual con LayerNorm escrita a mano), red feed-forward con ReLU y otro *Add & Norm*. La predicción sale del vector de la **posición 0**, que actúa como token CLS y debe reunir, a través de la atención, la información de toda la ventana.

3.3 Entrenamiento y resultados

Se entrenó con $d_{\text{model}} = 32$, 2 cabezas y $d_{\text{ff}} = 64$, en las mismas condiciones que el LSTM (20 épocas, Adam, *batch size* 64, MSE y predicción residual).

Modelo	H	MAE	RMSE	MAE en °C	Ratio vs naive
Transformer	24 h	0.1933	0.2510	1.66	0.968
Transformer	48 h	0.2475	0.3070	2.12	0.950

Tabla 3. Métricas del Transformer en validación.

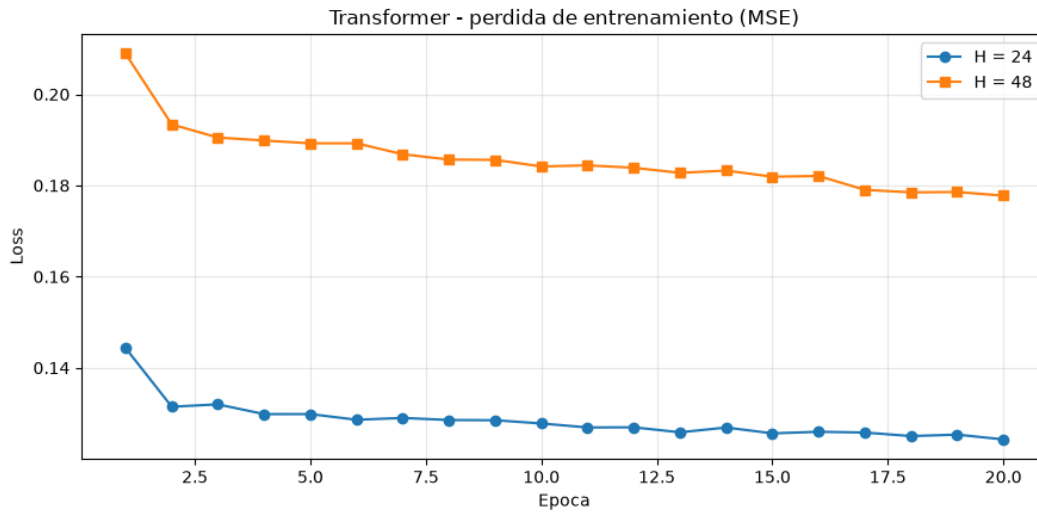


Figura 4. Pérdida de entrenamiento (MSE) del Transformer por época.

Para analizar la atención se tomaron 5 ejemplos de validación y se promediaron sus pesos sobre los ejemplos y las 2 cabezas. La fila t del mapa indica a qué posiciones miró la posición t :

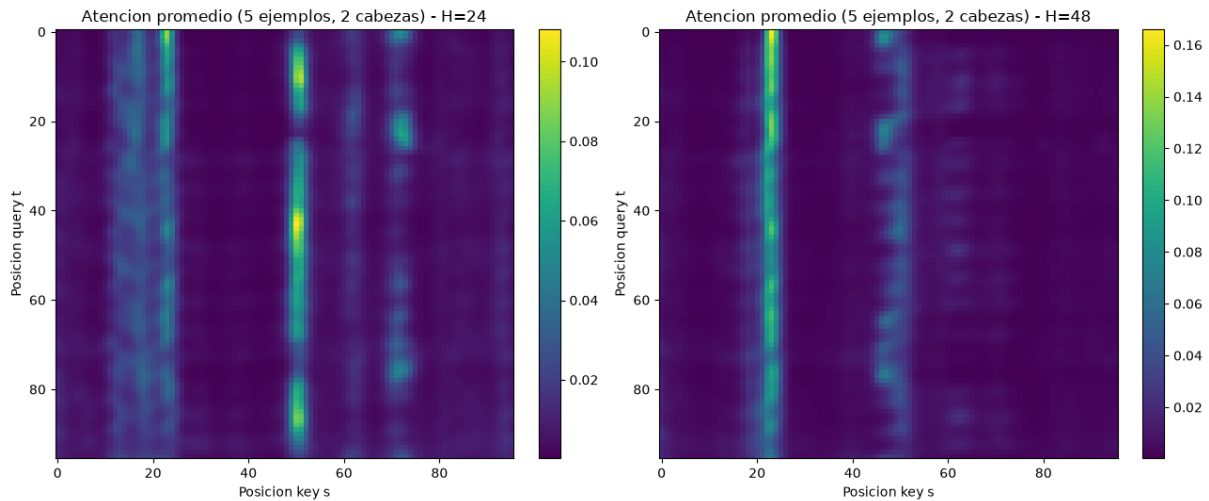


Figura 5. Mapa de atención promedio (96×96) para cada horizonte. Las columnas brillantes son posiciones que reciben atención desde casi todas las demás.

En ambos mapas aparecen columnas verticales definidas: casi todas las posiciones miran a los mismos pocos momentos de la ventana. La verificación oficial del Task 3 pasó correctamente: las formas de la salida y de la atención son las esperadas y cada fila de pesos suma 1.

3.3d. Identifique cuáles posiciones de la secuencia reciben mayor atención desde la posición 0 (el token CLS). Expresé esas posiciones como lags respecto al final de la ventana (lag 1 = el valor más reciente, lag 96 = el valor más antiguo).

La posición p corresponde al lag $96 - p$. Como referencia, una atención repartida por igual daría $1/96 \approx 0.010$ a cada lag.

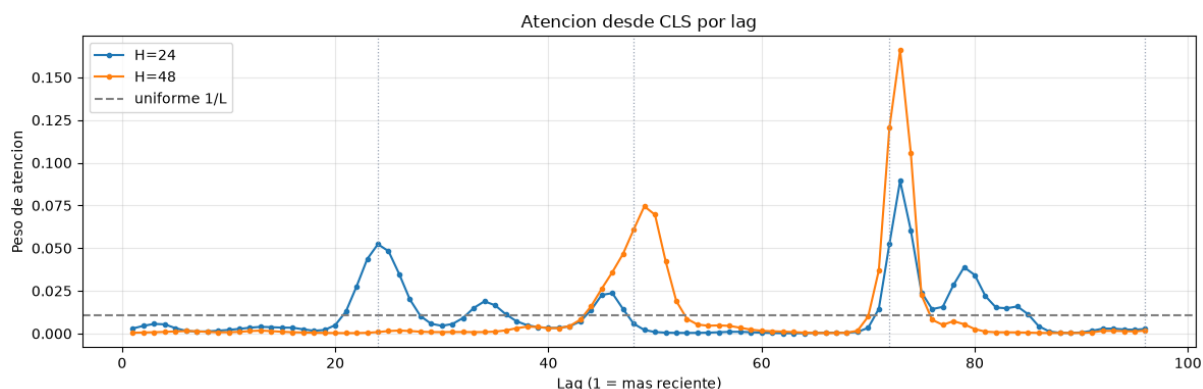


Figura 6. Atención desde el token CLS en función del lag. La línea discontinua es el nivel uniforme $1/96$.

#	H = 24: lag	Peso	H = 48: lag	Peso
1	73	0.089	73	0.166
2	74	0.061	72	0.121
3	72	0.053	74	0.106
4	24	0.052	49	0.074
5	25	0.048	50	0.070
6	23	0.044	48	0.061
7	79	0.039	47	0.047
8	26	0.035	51	0.042

Tabla 4. Lags con mayor atención desde el token CLS.

- **H = 24:** la atención se agrupa alrededor de los lags **72–74** (unos tres días atrás) y **23–26** (un día atrás), con un grupo secundario en 78–80.
- **H = 48:** la atención se concentra aún más, en los lags **72–74** y **46–51** (dos días atrás). El 83% del peso total está a menos de tres horas de un múltiplo de 24.
- **Lo más reciente casi no recibe atención:** los lags 1 a 3 suman 0.013 con $H = 24$ y 0.002 con $H = 48$.

En resumen, desde el token CLS el modelo mira sobre todo **la misma hora del día en días anteriores**: descubrió por sí solo el ciclo diario que la ACF había mostrado en el Task 1.

Task 4 · Análisis comparativo

4.1 ACF contra atención

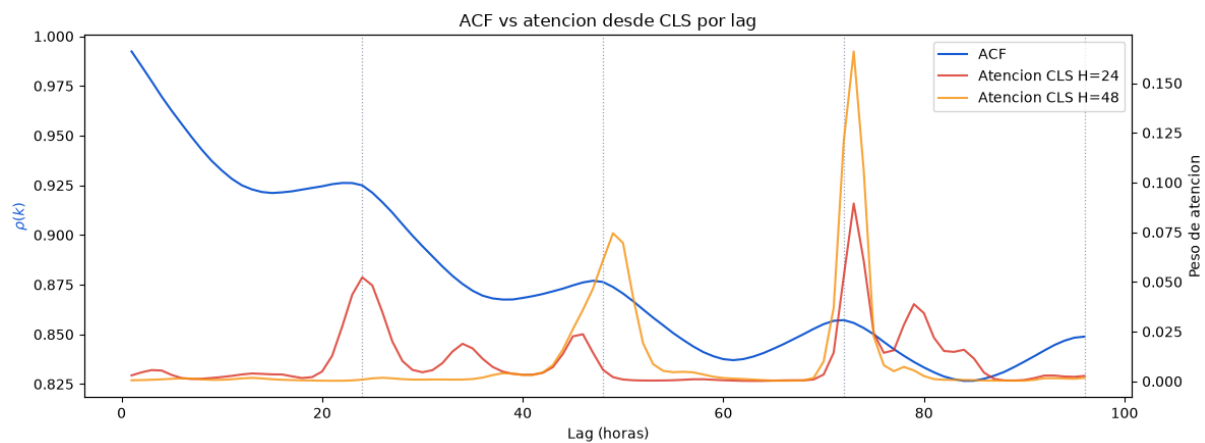


Figura 7. ACF (eje izquierdo) y atención desde CLS para ambos horizontes (eje derecho), por lag.

4.1a. ¿Los lags con mayor atención coinciden con los lags donde la ACF tiene sus picos más altos? Muestre los valores numéricos de ambos. Si no coinciden, proponga una hipótesis sobre por qué el Transformer aprendió un patrón de atención distinto al que la ACF sugiere.

Coinciden en parte. Los dos detectan el ciclo diario de la temperatura, pero no le dan importancia a los mismos momentos.

Lag (horas atrás)	ACF	Atención H = 24	Atención H = 48
1	0.992	0.003	0.001
2	0.985	0.004	0.001
24	0.925	0.052	0.001
48	0.876	0.006	0.061
72	0.857	0.053	0.121
73	0.856	0.089	0.166

Tabla 5. ACF y atención desde CLS en los lags clave (la atención uniforme sería 0.010).

En qué coinciden. La ACF tiene picos cerca de las 22, 47 y 72 horas, es decir, cada ~24 h. La atención también se concentra alrededor de múltiplos de 24 (lags 23–25, 47–50 y 72–74). Los dos métodos descubren lo mismo: la temperatura del aceite sigue un ciclo diario.

En qué no coinciden. Para la ACF, los lags más importantes son los más recientes (1 y 2 horas atrás, con correlación de 0.99), y el Transformer casi no los mira: reciben menos atención que si la repartiera por igual. Además, la ACF baja poco a poco con la distancia (hace un día > hace dos > hace tres), mientras que el Transformer prefiere el valor de **hace tres días** sobre el de hace uno. En toda la ventana, la correlación entre las dos curvas es prácticamente nula (−0.02 para H = 24 y −0.13 para H = 48).

Hipótesis de por qué el Transformer aprendió algo distinto:

1. **Lo reciente ya lo tiene incluido.** El modelo no predice la temperatura, sino cuánto va a cambiar respecto al último valor, que después se suma. La información de la última hora ya está en la predicción, así que mirarla otra vez no aporta. Lo útil para predecir el *cambio* es lo que pasó a esa misma hora en días anteriores.
2. **La ACF y el modelo responden preguntas distintas.** La ACF mide cuánto se parece el valor de *ahora* a los del pasado, pero el modelo debe predecir el valor de *dentro de 24 o 48 horas*. Ese momento futuro cae a la misma hora del día que hace 24, 48 y 72 horas, y justo ahí mira el modelo.
3. **No necesita mirar todo lo que se parece.** Las horas vecinas son casi iguales entre sí, así que la ACF de todas es alta. La atención funciona como una selección: le basta con elegir unos pocos momentos representativos en lugar de repartir peso entre muchos lags que dicen lo mismo.
4. **El ciclo no es exacto.** El pico de la ACF está en 22 horas y no en 24, y la atención cae en 73 y no en 72. El ciclo real no dura exactamente un día, y además los mapas son el promedio de solo 5 ejemplos.

4.1b. La ACF es una medida lineal de dependencia; el mecanismo de atención es no lineal porque los scores dependen de proyecciones aprendidas de los valores. Explique qué tipo de dependencia entre pasos temporales podría capturar el Transformer que la ACF no puede detectar, y si sus mapas de atención muestran alguna evidencia de ese tipo de dependencia.

La ACF da **un solo número por lag para toda la serie**: por ejemplo, que en promedio el valor de hace 24 horas se parece en 0.925 al actual. Ese número es fijo, promedia todos los días y solo compara pares de valores mediante una relación en línea recta. La atención, en cambio, **decide a qué momentos mirar en cada ventana según lo que ve en ella**. Por eso puede captar relaciones que la ACF no detecta:

- **Dependencias que cambian según la situación.** Quizá el valor de hace 24 horas importa mucho cuando la temperatura viene subiendo y poco cuando está estable, o se comporta distinto en fin de semana o en verano. La ACF mezcla todos esos casos en un solo promedio.
- **Combinaciones de varios momentos.** Por ejemplo: si subió rápido en las últimas horas y ayer a esta hora hubo un pico, entonces... La ACF solo compara dos momentos a la vez.
- **Relaciones que no son en línea recta**, como efectos de umbral (a partir de cierta temperatura se activa el enfriamiento) o de saturación. La correlación lineal puede ser baja aunque la relación sea muy útil para predecir.
- **Búsqueda de patrones parecidos:** la atención puede ir a un momento del pasado que se parezca a la situación actual, aunque ese lag tenga una ACF baja.

Evidencia en nuestros mapas:

1. **El patrón cambia según lo que se quiere predecir.** La ACF es una sola, pero la atención es distinta en cada modelo: con $H = 24$ mira las horas 23–26 y 72–74 atrás, y con $H = 48$ se mueve a 46–51 y 72–74. El Transformer adapta a dónde mira según el objetivo, algo que la ACF no puede hacer.
2. **Da peso a momentos que la ACF considera menos importantes.** Los lags 78–80 (ACF de 0.83) reciben más atención que los lags 1–3 (ACF de 0.98).

3. **Limitación:** se analizó el *promedio* de 5 ejemplos. Para demostrar que la atención cambia de una ventana a otra según su contenido habría que comparar los mapas ejemplo por ejemplo. Con el promedio se puede afirmar que el patrón aprendido difiere del de la ACF, pero no probar del todo que dependa de cada situación.

4.2 LSTM contra Transformer

Modelo	Horizonte	MAE	RMSE	Ratio vs naive
LSTM	24 h	0.1959	0.2602	0.982
LSTM	48 h	0.2389	0.3096	0.918
Transformer	24 h	0.1933	0.2510	0.968
Transformer	48 h	0.2475	0.3070	0.950
Naive	24 h	0.1996	0.2672	1.000
Naive	48 h	0.2604	0.3359	1.000

Tabla 6. Comparación en validación con una corrida por modelo (escala normalizada).

Experimento con más semillas

Las diferencias entre modelos en la Tabla 6 son de apenas 1–4%. Para saber si son reales o dependen de la semilla aleatoria (inicialización de pesos y orden de los mini-batches), se repitió el entrenamiento con 6 semillas (0, 1, 2, 3, 4 y 42), con la misma configuración y en modo determinista para que el experimento sea reproducible.

Modelo	H	MAE (media ± desv.)	Ratio vs naive	Transformer gana en
LSTM	24 h	0.1986 ± 0.0044	0.995 ± 0.022	
Transformer	24 h	0.1954 ± 0.0049	0.979 ± 0.025	4 de 6 semillas
LSTM	48 h	0.2515 ± 0.0027	0.966 ± 0.010	
Transformer	48 h	0.2429 ± 0.0044	0.933 ± 0.017	6 de 6 semillas

Tabla 7. Resultados con 6 semillas.

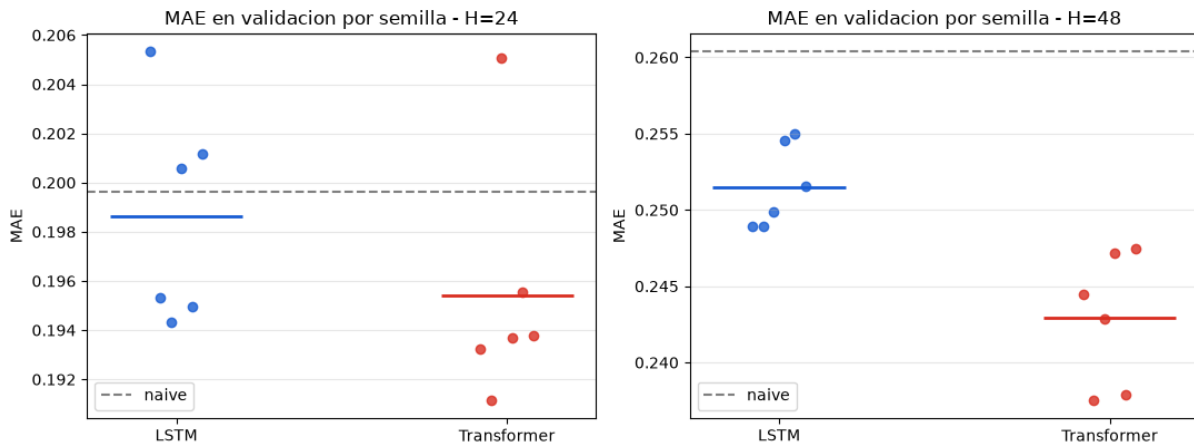


Figura 8. MAE en validación de cada semilla. Las líneas horizontales son la media de cada modelo y la línea discontinua, el naive.

Con 48 horas el Transformer supera al LSTM en todas las semillas. Con 24 horas la diferencia media es más pequeña que la variación entre semillas, así que ambos modelos están empatados. El resultado del LSTM con 48 horas en la corrida única (0.2389) resultó atípico: queda fuera del rango observado con las seis semillas (0.2489 a 0.2550).

4.2a. Al aumentar el horizonte de 24 a 48 horas, el error de ambos modelos aumenta. Explique matemáticamente por qué ese aumento es esperado en el caso del LSTM usando la fórmula de acumulación de error en predicción iterativa. En el caso del direct multi-output, el error no se acumula iterativamente: ¿por qué también aumenta con el horizonte?

Al pasar de 24 a 48 horas, el MAE aumenta 22% en el LSTM, 28% en el Transformer y 30% en el naive. La idea general es la misma que con el pronóstico del clima: el de mañana es bastante confiable, el de dentro de una semana, mucho menos.

Caso iterativo. Un modelo iterativo predice solo una hora adelante y usa esa predicción como si fuera un dato real para predecir la siguiente. Cada paso comete un error nuevo y además **hereda los errores de los pasos anteriores**. Para una serie en la que cada valor depende del anterior con un factor ϕ , la varianza del error después de h pasos es:

$$\text{Var}(\text{error}_h) = \sigma^2 (1 + \phi^2 + \phi^4 + \dots + \phi^{2(h-1)})$$

Cada paso adicional suma un término positivo, así que el error solo puede crecer: con 48 pasos se acumulan más términos que con 24. En este laboratorio, sin embargo, **ambos modelos son directos**: predicen el valor de dentro de H horas en un solo paso, así que esta acumulación describe lo que pasaría con un LSTM iterativo.

Caso directo. Aquí no hay acumulación, pero el error crece por una razón más básica: **cuanto más lejos está el futuro, menos información da el presente sobre él**. En 48 horas pasan más cosas que la ventana no puede anticipar (cambios de carga, clima u operación del transformador). La ACF lo muestra: el valor actual se parece en 0.925 al de dentro de 24 horas, pero solo en 0.876 al de dentro de 48. Esa diferencia, que parece pequeña, hace que el error del naive en entrenamiento suba de 0.150 a 0.249, un 65% más. Los modelos aprenden a corregir al naive, pero no pueden eliminar esa incertidumbre adicional.

4.2b. Si el Transformer supera al LSTM en horizonte $H = 48$ pero no en $H = 24$, proponga una explicación basada en la diferencia arquitectónica entre ambos modelos respecto al flujo del gradiente durante el entrenamiento. Use la fórmula del gradiente en el LSTM y en el Transformer.

Con una sola corrida (Tabla 6) la premisa no se observa: el Transformer gana con 24 horas y el LSTM con 48. Pero el experimento con 6 semillas (Tabla 7) muestra que **la premisa sí se cumple en lo esencial**: el Transformer gana siempre con 48 horas y los modelos empatan con 24. La explicación está en el camino que recorre el gradiente: para aprender, el error de la predicción debe viajar hacia atrás hasta cada hora de la ventana.

En el LSTM ese camino es como un teléfono descompuesto. La predicción sale solo del último estado, así que para llegar a la hora de hace 72 horas el error debe atravesar los 72 pasos intermedios. En cada uno se multiplica por la compuerta de olvido, un valor menor que 1:

$$\partial L / \partial c_k \approx \partial L / \partial c_T \cdot f_{k+1} \cdot f_{k+2} \cdot \dots \cdot f_T$$

Multiplicar muchas veces por números menores que 1 da un número muy pequeño: la señal llega débil a las horas lejanas y al modelo le cuesta aprender de ellas.

En el Transformer el camino es una llamada directa. La salida del token CLS es un promedio ponderado de todas las horas, así que el error llega a cada una **en un solo salto**, y su intensidad depende solo del peso de atención, no de la distancia:

$$\partial z_o / \partial v_s = \alpha_{os}$$

Por qué la ventaja aparece con 48 horas y no con 24. Con $H = 48$ la información útil está lejos: según los mapas, el modelo mira sobre todo las horas de hace 46–51 y 72–74. Para el LSTM eso es un camino largo; para el Transformer es la misma llamada directa de siempre. Con $H = 24$ también sirven momentos más cercanos (hace 23–26 horas), y el valor más reciente ya entra directamente en la predicción, así que la desventaja del LSTM pesa menos. El LSTM no pierde por mucho porque su celda de memoria está diseñada justamente para mitigar este problema: si la compuerta de olvido se mantiene cerca de 1, la señal casi no se debilita. Por eso la ventaja del Transformer es moderada (~3.4% de MAE) y no enorme.

Queda también una lección metodológica: cuando las diferencias entre modelos son de 1–4%, una sola corrida puede llevar a la conclusión contraria. Aquí la corrida única decía lo opuesto de lo que muestran seis semillas.

Conclusiones

- La temperatura del aceite es una serie muy persistente, con un **ciclo diario** claro ($\rho(24) \approx 0.925$) que justifica una ventana de 96 horas.
- Por el cambio de nivel entre periodos, predecir la temperatura directamente generaliza mal. La **predicción residual** (aprender el cambio respecto al último valor) permitió que ambos modelos superaran al baseline naïve.
- El Transformer aprendió por sí solo a mirar la **misma hora del día en días anteriores**, en coincidencia con la periodicidad de la ACF, aunque con un orden de importancia distinto.

- Con varias semillas, el Transformer supera de forma consistente al LSTM con $H = 48$, y ambos empatan con $H = 24$, en línea con la diferencia en el flujo del gradiente entre ambas arquitecturas.
- Con diferencias de pocos puntos porcentuales, comparar modelos exige repetir el entrenamiento con varias semillas.